Licenciatura em Engenharia Informática
Universidade do Minho

# REDES DE COMPUTADORES

**2025/2026**

## Trabalho Prático 2 - Desenvolvimento de um Packet Sniffer e Analisador de tráfego

**Realizado por:**

Matilde Rodrigues Teixeira - A106876
Francisco Quintas Barros - A106894
José Luís Matos Ribeiro - A105108

2026

# Índice

**1 Introdução** **4**

**2 Arquitetura e funcionamento do sniffer** **5**
2.1 Captura de pacotes . . . . . 5
2.2 Processamento e identificação de protocolos . . . . . 5
2.3 Correlação de mensagens . . . . . 5
2.4 Filtragem de tráfego . . . . . 6
2.5 Apresentação e registo dos dados . . . . . 6
2.6 Geração de gráficos . . . . . 6
2.7 Menu interativo e execução por linha de comandos . . . . . 6

**3 Parte A** **7**
3.1 Topologia de Rede . . . . . 7
3.2 Captura geral sem filtro . . . . . 8
3.3 Teste ICMP com ping . . . . . 8
3.4 Teste ARP . . . . . 9
3.5 Teste IPv4 . . . . . 10
3.6 Teste DHCP . . . . . 11
3.7 Conclusão Parte A . . . . . 11

**4 Parte B** **12**
4.1 Teste ARP na interface real . . . . . 12
4.2 Filtragem de tráfego ICMP . . . . . 13
4.3 Teste IPv4 na interface real . . . . . 14
4.4 Teste DHCP na interface real . . . . . 15
4.5 Captura de tráfego sem filtros . . . . . 16
4.6 Geração de gráficos . . . . . 17
4.7 Conclusão Parte B . . . . . 18

**5 Conclusão** **19**

# List of Figures


1 Topologia de rede utilizada no CORE . . . 7
2 Captura geral de múltiplos protocolos . . . 8
3 Resultado do comando ping entre Castelo e Afonso Henriques . . . 8
4 Captura de pacotes ICMP com filtro aplicado . . . 9
5 Execução do comando ping para gerar tráfego ARP . . . 9
6 Captura de pacotes ARP no sniffer . . . 10
7 Execução do comando ping para gerar tráfego IPv4 . . . 10
8 Captura de pacotes IPv4 no sniffer . . . 11
9 Captura de mensagens DHCP Discover . . . 11
10 Execução do comando dhclient . . . 11
11 Captura de pacotes ARP na interface real . . . 12
12 Execução do comando ping para 8.8.8.8 . . . 12
13 Captura de tráfego ICMP com filtro aplicado . . . 13
14 Execução do comando ping para 8.8.8.8 . . . 13
15 Captura de mensagens IPv4 com filtro aplicado . . . 14
16 Execução do comando ping para 8.8.8.8 . . . 15
17 Captura de mensagens DHCP . . . 15
18 Captura de tráfego sem filtros na interface real . . . 16
19 Gráfico de distribuição de pacotes por protocolo . . . 17
20 Gráfico de RTT dos *replies* ICMP ao longo do tempo . . . 17
21 Distribuição do tamanho dos pacotes capturados . . . 18
22 Número de pacotes capturados por segundo . . . 18

# 1 Introdução

O presente relatório foi elaborado no âmbito da Unidade Curricular de Redes de Computadores e tem como objetivo documentar o desenvolvimento e validação de um *packet sniffer* para captura e análise de tráfego de rede.

Neste trabalho, o foco incide na identificação, classificação e análise de protocolos relevantes nas várias camadas do modelo TCP/IP, nomeadamente ARP, IPv4, ICMP, DHCP, DNS e HTTP/HTTPS. Para cada protocolo, são analisados os campos relevantes dos respetivos cabeçalhos, bem como os padrões de interação observados entre entidades da rede.

A validação do sniffer foi realizada em dois ambientes distintos: primeiro num ambiente emulado no CORE, onde foi possível gerar tráfego controlado e isolar o comportamento de cada protocolo; e depois numa interface de rede real, onde se observou tráfego mais diversificado e menos previsível. Em ambos os casos, o objetivo foi relacionar os pacotes capturados com o comportamento esperado dos protocolos analisados.

O objetivo final é demonstrar não só o correto funcionamento da ferramenta desenvolvida, mas também a capacidade de interpretar o tráfego capturado e validar os mecanismos de captura, filtragem, logging, correlação e análise implementados.

# 2 Arquitetura e funcionamento do sniffer

O sniffer desenvolvido consiste numa aplicação em Python responsável pela captura, processamento, filtragem e apresentação de pacotes de rede em tempo real. A sua arquitetura segue uma abordagem modular, separando as diferentes fases do processamento de tráfego em ficheiros distintos: `capture.py` (captura e correlação), `parser.py` (interpretação dos pacotes), `filters.py` (filtragem), `output.py` (apresentação e logs) e `plots.py` (geração de gráficos).

## 2.1 Captura de pacotes

A captura de pacotes é realizada diretamente a partir de uma interface de rede selecionada pelo utilizador, recorrendo a permissões de baixo nível (*root*) para acesso aos frames, através da biblioteca Scapy. O sniffer suporta execução tanto no ambiente emulado CORE como em interfaces reais, como Ethernet ou Wi-Fi.

Os pacotes são capturados de forma contínua, sendo possível limitar o número de pacotes capturados através do parâmetro `--count` ou executar a captura indefinidamente em modo *live*. Esta abordagem permite observar o tráfego em tempo real e analisar o comportamento dinâmico dos protocolos.

## 2.2 Processamento e identificação de protocolos

Após a captura, cada pacote é analisado através do *parsing* dos cabeçalhos das diferentes camadas do modelo TCP/IP, recorrendo à biblioteca Scapy. O sniffer verifica a presença das várias camadas e identifica automaticamente os protocolos envolvidos, nomeadamente:

- ARP;
- IPv4 e IPv6;
- ICMP, TCP e UDP;
- DHCP, DNS e HTTP/HTTPS.

Para cada pacote, são extraídos campos relevantes como endereços MAC e IP, portas de origem e destino, tamanho do pacote e tipo de mensagem. Para além disso, são analisados campos específicos de cada protocolo, permitindo distinguir diferentes tipos de mensagens. Por exemplo, no caso do ICMP são identificados os tipos *echo request* e *echo reply*; no DHCP são analisadas as opções do protocolo para reconhecer mensagens *Discover*, *Offer*, *Request* e *Acknowledge*; e no DNS é extraído o nome da query e identificado se o pacote corresponde a uma pergunta ou resposta.

Note-se que o tráfego na porta 443, embora associado a tráfego web, corresponde na prática a HTTPS sobre TLS. Assim, o conteúdo aplicacional encontra-se cifrado, sendo apenas possível analisar metadados como endereços, portas e direção da comunicação.

## 2.3 Correlação de mensagens

Para além da análise individual de pacotes, o sniffer implementa um mecanismo de correlação entre mensagens de pedido (*request*) e resposta (*reply*), permitindo identificar interações completas entre entidades da rede. Este mecanismo é suportado para os seguintes protocolos:

- **ICMP** — correlação por endereço IP, identificador e número de sequência do pacote;
- **ARP** — correlação por endereços IP de origem e destino;
- **DHCP** — correlação pelo endereço MAC do cliente e pelo campo `xid`;
- **DNS** — correlação por endereços IP, portas e *transaction ID*.

Quando um pedido é capturado, o seu identificador é armazenado internamente, sendo posteriormente utilizado para associar a resposta correspondente. Para os *replies* ICMP, é ainda calculado o RTT (*Round Trip Time*) em milissegundos, como diferença entre o timestamp do *request* e o do *reply* correspondente. Esta informação é apresentada no output e guardada no log.

## 2.4 Filtragem de tráfego

O sniffer suporta múltiplos mecanismos de filtragem, permitindo restringir a captura a subconjuntos específicos de tráfego. Entre os filtros disponíveis destacam-se:

- filtragem por protocolo (`--proto`, ex.: ICMP, ARP, DHCP, IPv4);
- filtragem por endereço IP (`--ip`);
- filtragem por endereço MAC (`--mac`);
- expressões BPF (`--bpf`, ex.: `icmp or arp`);
- filtragem por campos de cabeçalho (`--hfilter`, ex.: `ip.src==10.0.0.1 and icmp.type==8`).

Os filtros por campos permitem definir condições mais específicas utilizando operadores lógicos e operadores de comparação. Cada pacote capturado é avaliado de acordo com estas expressões, sendo apresentado apenas se satisfizer as condições definidas.

## 2.5 Apresentação e registo dos dados

O sniffer pode operar em três modos principais:

- **Modo live** — os pacotes são apresentados em tempo real na consola;
- **Modo log** — os dados são guardados em ficheiros nos formatos `.json`, `.csv` ou `.txt`;
- **Modo combinado** — live e log em simultâneo.

Cada pacote capturado é associado a um identificador sequencial, permitindo a sua referência durante a análise. O campo *summary* apresenta um resumo estruturado da informação mais relevante, incluindo tipo de mensagem, endereços, portas, nível da pilha protocolar e RTT quando aplicável.

## 2.6 Geração de gráficos

Após a captura, o sniffer pode gerar automaticamente gráficos em formato PNG, guardados na diretoria `logs/plots/`. Os gráficos disponíveis incluem:

- distribuição dos pacotes capturados por protocolo;
- número de pacotes por segundo ao longo do tempo;
- evolução do RTT dos *replies* ICMP ao longo do tempo;
- distribuição do tamanho dos pacotes.

Esta funcionalidade permite complementar a análise textual com uma visão gráfica do tráfego capturado.

## 2.7 Menu interativo e execução por linha de comandos

A execução do sniffer pode ser feita diretamente por linha de comandos, especificando os parâmetros desejados, ou através de um menu interativo em terminal (`--menu`).

No menu interativo, o utilizador é guiado passo a passo para configurar a interface de rede, o modo de saída, os filtros, os gráficos a gerar e o número de pacotes a capturar. O menu apresenta ainda um resumo das configurações antes de iniciar a captura, permitindo verificar os parâmetros selecionados.

# 3 Parte A

Nesta secção são apresentados os testes realizados no ambiente de rede CORE, com o objetivo de validar o correto funcionamento do packet sniffer desenvolvido.

Pretende-se verificar a capacidade da aplicação para capturar pacotes, identificar corretamente os protocolos envolvidos e aplicar mecanismos de filtragem sobre o tráfego observado.

Para tal, foram realizados vários testes práticos que permitem gerar e analisar tráfego de diferentes protocolos, nomeadamente ARP, ICMP, IPv4 e DHCP, bem como capturas contendo múltiplos tipos de pacotes em simultâneo. Estes testes permitem observar as interações entre entidades da rede, como pedidos e respostas, confirmando a correta interpretação dos protocolos pelo sniffer.

## 3.1 Topologia de Rede

De forma a validar o funcionamento do sniffer num ambiente controlado, foi utilizada a topologia de rede do TP1, apresentada na Figura 1.

Esta topologia é composta por vários domínios de rede interligados através de routers e switches, contendo múltiplas sub-redes e nós finais. Esta configuração permite gerar diferentes tipos de tráfego, incluindo comunicação local dentro da mesma sub-rede e comunicação entre redes distintas.

Esta estrutura permite observar tráfego intra-sub-rede e inter-sub-redes, incluindo encaminhamento através de routers, o que é relevante para validar a interpretação de protocolos ao nível da camada de rede.

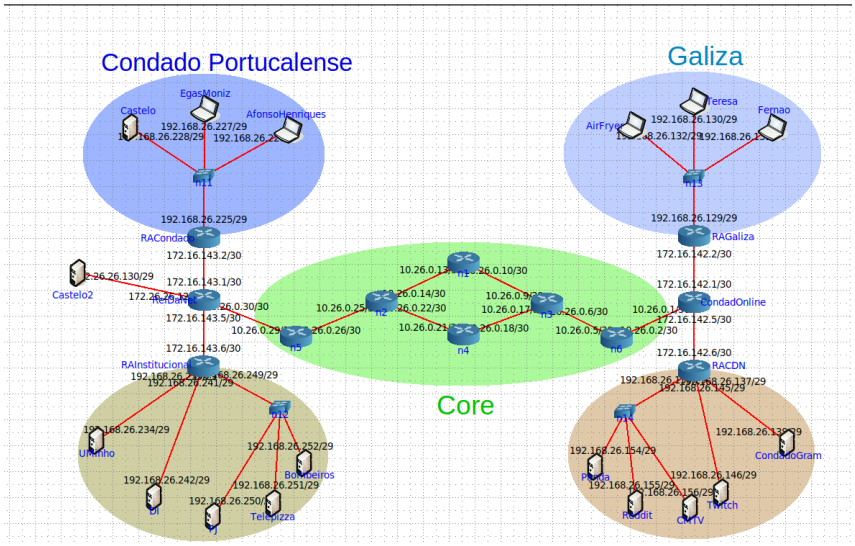



Figure 1: Topologia de rede utilizada no CORE

O sniffer foi executado num dos nós da topologia, na interface `eth0`, capturando o tráfego gerado localmente e entre diferentes sub-redes. Esta configuração permite observar protocolos como ARP, IPv4, ICMP e DHCP, sendo adequada para validar a capacidade de captura e análise da aplicação.

## 3.2 Captura geral sem filtro

```
(.venv) root@AfonsoHenriques:/home/core/Desktop/RC# sudo ./.venv/bin/python sniffer.py --iface eth0 --live --count 20
sudo: unable to resolve host AfonsoHenriques: Temporary failure in name resolution
WARNING: Could not retrieve the OS's nameserver !
TIME       SOURCE             DESTINATION        PROTO    INFO
--------------------------------------------------------------------------------
15:45:34   192.168.26.225     224.0.0.5          IPv4     capture(id=1) 192.168.26.225 -> 224.0.0.5 | Nível=3 | IPv4 packet
15:45:36   192.168.26.225     224.0.0.5          IPv4     capture(id=2) 192.168.26.225 -> 224.0.0.5 | Nível=3 | IPv4 packet
15:45:38   192.168.26.225     224.0.0.5          IPv4     capture(id=3) 192.168.26.225 -> 224.0.0.5 | Nível=3 | IPv4 packet
15:45:40   192.168.26.225     224.0.0.5          IPv4     capture(id=4) 192.168.26.225 -> 224.0.0.5 | Nível=3 | IPv4 packet
15:45:41   fe80::200:ff:feaa:18 ff02::5           IPv6     capture(id=5) fe80::200:ff:feaa:18 -> ff02::5 | Nível=3 | IPv6 packet
15:45:42   192.168.26.225     224.0.0.5          IPv4     capture(id=6) 192.168.26.225 -> 224.0.0.5 | Nível=3 | IPv4 packet
15:45:44   192.168.26.225     224.0.0.5          IPv4     capture(id=7) 192.168.26.225 -> 224.0.0.5 | Nível=3 | IPv4 packet
15:45:46   192.168.26.225     224.0.0.5          IPv4     capture(id=8) 192.168.26.225 -> 224.0.0.5 | Nível=3 | IPv4 packet
15:45:47   192.168.26.228     192.168.26.226     ICMP     request(id=9) 192.168.26.228 -> 192.168.26.226 | Nível=4 | ICMP echo request
15:45:47   192.168.26.226     192.168.26.228     ICMP     reply(id=10) ao request(id=9) 192.168.26.226 -> 192.168.26.228 | Nível=4 | ICMP ec
ho reply
15:45:48   192.168.26.228     192.168.26.226     ICMP     request(id=11) 192.168.26.228 -> 192.168.26.226 | Nível=4 | ICMP echo request
15:45:48   192.168.26.226     192.168.26.228     ICMP     reply(id=12) ao request(id=11) 192.168.26.226 -> 192.168.26.228 | Nível=4 | ICMP e
cho reply
15:45:48   192.168.26.225     224.0.0.5          IPv4     capture(id=13) 192.168.26.225 -> 224.0.0.5 | Nível=3 | IPv4 packet
15:45:49   192.168.26.228     192.168.26.226     ICMP     request(id=14) 192.168.26.228 -> 192.168.26.226 | Nível=4 | ICMP echo request
15:45:49   192.168.26.226     192.168.26.228     ICMP     reply(id=15) ao request(id=14) 192.168.26.226 -> 192.168.26.228 | Nível=4 | ICMP e
cho reply
15:45:50   192.168.26.228     192.168.26.226     ICMP     request(id=16) 192.168.26.228 -> 192.168.26.226 | Nível=4 | ICMP echo request
15:45:50   192.168.26.226     192.168.26.228     ICMP     reply(id=17) ao request(id=16) 192.168.26.226 -> 192.168.26.228 | Nível=4 | ICMP e
cho reply
15:45:50   192.168.26.225     224.0.0.5          IPv4     capture(id=18) 192.168.26.225 -> 224.0.0.5 | Nível=3 | IPv4 packet
15:45:51   192.168.26.228     192.168.26.226     ICMP     request(id=19) 192.168.26.228 -> 192.168.26.226 | Nível=4 | ICMP echo request
15:45:51   192.168.26.226     192.168.26.228     ICMP     reply(id=20) ao request(id=19) 192.168.26.226 -> 192.168.26.228 | Nível=4 | ICMP e
cho reply

--- Estatísticas ---
ICMP     : 10
IPv4     : 9
IPv6     : 1
```

Figure 2: Captura geral de múltiplos protocolos

Nesta captura o sniffer foi executado sem qualquer filtro, com o objetivo de observar múltiplos protocolos em simultâneo.

Como se verifica na Figura 2, são capturados diferentes tipos de tráfego, incluindo pacotes IPv4 com destino multicast `224.0.0.5`, mensagens ICMP associadas à execução do comando *ping* e ainda pacotes IPv6.

A presença destes diferentes tipos de pacotes demonstra que o sniffer consegue identificar e classificar tráfego heterogéneo. Em particular, a identificação de tráfego multicast e de pacotes não gerados explicitamente, como IPv6, evidencia que a aplicação capta o tráfego presente na interface e o classifica corretamente.

No final da execução, o sniffer apresenta estatísticas por protocolo, confirmando a classificação dos pacotes observados.

## 3.3 Teste ICMP com ping

Neste teste foi aplicado um filtro ao protocolo ICMP no sniffer e, em simultâneo, foi executado o comando *ping* entre os nós Castelo e Afonso Henriques, com o objetivo de validar a captura das mensagens geradas por este protocolo.

```
root@Castelo:/tmp/pycore.37441/Castelo.conf# ping 192.168.26.226
PING 192.168.26.226 (192.168.26.226) 56(84) bytes of data.
64 bytes from 192.168.26.226: icmp_seq=1 ttl=64 time=0.024 ms
64 bytes from 192.168.26.226: icmp_seq=2 ttl=64 time=0.045 ms
64 bytes from 192.168.26.226: icmp_seq=3 ttl=64 time=0.027 ms
64 bytes from 192.168.26.226: icmp_seq=4 ttl=64 time=0.046 ms
64 bytes from 192.168.26.226: icmp_seq=5 ttl=64 time=0.026 ms
64 bytes from 192.168.26.226: icmp_seq=6 ttl=64 time=0.027 ms
64 bytes from 192.168.26.226: icmp_seq=7 ttl=64 time=0.085 ms
^C
--- 192.168.26.226 ping statistics ---
7 packets transmitted, 7 received, 0% packet loss, time 6137ms
rtt min/avg/max/mdev = 0.024/0.040/0.085/0.020 ms
root@Castelo:/tmp/pycore.37441/Castelo.conf#
```

Figure 3: Resultado do comando ping entre Castelo e Afonso Henriques

Como se observa na Figura 3, a comunicação entre os dois nós foi bem-sucedida, apresentando 0% de perda de pacotes. Os valores de RTT são reduzidos, como esperado num ambiente emulado no CORE.

```
(.venv) root@AfonsoHenriques:/home/core/Desktop/RC# sudo ./.venv/bin/python sniffer.py --iface eth0 --proto ICMP --live --count 10
sudo: unable to resolve host AfonsoHenriques: Temporary failure in name resolution
WARNING: Could not retrieve the OS's nameserver !
TIME       SOURCE            DESTINATION       PROTO    INFO
--------------------------------------------------------------------------------
15:21:45   192.168.26.228    192.168.26.226    ICMP     request(id=1) 192.168.26.228 -> 192.168.26.226 | Nível=4 | ICMP echo request
15:21:45   192.168.26.226    192.168.26.228    ICMP     reply(id=2) ao request(id=1) 192.168.26.226 -> 192.168.26.228 | Nível=4 | ICMP ech
o reply
15:21:46   192.168.26.228    192.168.26.226    ICMP     request(id=4) 192.168.26.228 -> 192.168.26.226 | Nível=4 | ICMP echo request
15:21:46   192.168.26.226    192.168.26.228    ICMP     reply(id=5) ao request(id=4) 192.168.26.226 -> 192.168.26.228 | Nível=4 | ICMP ech
o reply
15:21:47   192.168.26.228    192.168.26.226    ICMP     request(id=6) 192.168.26.228 -> 192.168.26.226 | Nível=4 | ICMP echo request
15:21:47   192.168.26.226    192.168.26.228    ICMP     reply(id=7) ao request(id=6) 192.168.26.226 -> 192.168.26.228 | Nível=4 | ICMP ech
o reply
15:21:48   192.168.26.228    192.168.26.226    ICMP     request(id=9) 192.168.26.228 -> 192.168.26.226 | Nível=4 | ICMP echo request
15:21:48   192.168.26.226    192.168.26.228    ICMP     reply(id=10) ao request(id=9) 192.168.26.226 -> 192.168.26.228 | Nível=4 | ICMP ec
ho reply

--- Estatísticas ---
ICMP      : 8
```

Figure 4: Captura de pacotes ICMP com filtro aplicado

Na Figura 4, o sniffer capturou as mensagens *ICMP echo request* enviadas de `192.168.26.228` para `192.168.26.226`, bem como as respetivas mensagens *echo reply* no sentido inverso. Observa-se que cada pedido é seguido por uma resposta correspondente, sendo possível identificar a relação request/reply através dos identificadores apresentados pelo sniffer.

A correspondência entre o resultado do comando *ping* e os pacotes capturados confirma que o filtro ICMP funciona corretamente e que o sniffer interpreta o tráfego ICMP associado à comunicação entre os dois nós.

## 3.4 Teste ARP

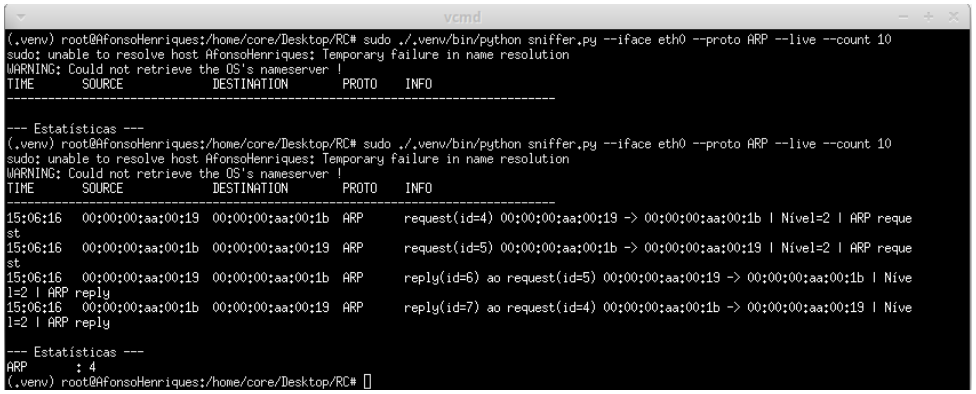
Para observar o funcionamento do protocolo ARP, foi gerado tráfego através da execução do comando *ping*, forçando a resolução de endereços na rede local.

```
vcmd

root@Castelo:/tmp/pycore.37441/Castelo.conf# ping 192.168.26.226
PING 192.168.26.226 (192.168.26.226) 56(84) bytes of data.
64 bytes from 192.168.26.226: icmp_seq=1 ttl=64 time=0.059 ms
64 bytes from 192.168.26.226: icmp_seq=2 ttl=64 time=0.069 ms
64 bytes from 192.168.26.226: icmp_seq=3 ttl=64 time=0.073 ms
64 bytes from 192.168.26.226: icmp_seq=4 ttl=64 time=0.023 ms
64 bytes from 192.168.26.226: icmp_seq=5 ttl=64 time=0.084 ms
64 bytes from 192.168.26.226: icmp_seq=6 ttl=64 time=0.072 ms
64 bytes from 192.168.26.226: icmp_seq=7 ttl=64 time=0.085 ms
64 bytes from 192.168.26.226: icmp_seq=8 ttl=64 time=0.073 ms
64 bytes from 192.168.26.226: icmp_seq=9 ttl=64 time=0.028 ms
64 bytes from 192.168.26.226: icmp_seq=10 ttl=64 time=0.074 ms
64 bytes from 192.168.26.226: icmp_seq=11 ttl=64 time=0.058 ms
64 bytes from 192.168.26.226: icmp_seq=12 ttl=64 time=0.070 ms
^C
--- 192.168.26.226 ping statistics ---
12 packets transmitted, 12 received, 0% packet loss, time 11236ms
rtt min/avg/max/mdev = 0.023/0.064/0.085/0.018 ms
root@Castelo:/tmp/pycore.37441/Castelo.conf#
```

Figure 5: Execução do comando ping para gerar tráfego ARP

Como se observa na Figura 5, foi estabelecida comunicação entre os dois nós, sem perda de pacotes. A execução do *ping* implica, numa fase inicial, a resolução do endereço MAC correspondente ao endereço IP de destino, recorrendo ao protocolo ARP.



```
vcmd
(.venv) root@AfonsoHenriques:/home/core/Desktop/RC# sudo ./.venv/bin/python sniffer.py --iface eth0 --proto ARP --live --count 10
sudo: unable to resolve host AfonsoHenriques: Temporary failure in name resolution
WARNING: Could not retrieve the OS's nameserver !
TIME       SOURCE            DESTINATION       PROTO    INFO
--------------------------------------------------------------------------------

--- Estatísticas ---
(.venv) root@AfonsoHenriques:/home/core/Desktop/RC# sudo ./.venv/bin/python sniffer.py --iface eth0 --proto ARP --live --count 10
sudo: unable to resolve host AfonsoHenriques: Temporary failure in name resolution
WARNING: Could not retrieve the OS's nameserver !
TIME       SOURCE            DESTINATION       PROTO    INFO
--------------------------------------------------------------------------------
15:06:16   00:00:00:aa:00:19  00:00:00:aa:00:1b  ARP      request(id=4) 00:00:00:aa:00:19 -> 00:00:00:aa:00:1b | Nível=2 | ARP reque
st
15:06:16   00:00:00:aa:00:1b  00:00:00:aa:00:19  ARP      request(id=5) 00:00:00:aa:00:1b -> 00:00:00:aa:00:19 | Nível=2 | ARP reque
st
15:06:16   00:00:00:aa:00:19  00:00:00:aa:00:1b  ARP      reply(id=6) ao request(id=5) 00:00:00:aa:00:19 -> 00:00:00:aa:00:1b | Níve
l=2 | ARP reply
15:06:16   00:00:00:aa:00:1b  00:00:00:aa:00:19  ARP      reply(id=7) ao request(id=4) 00:00:00:aa:00:1b -> 00:00:00:aa:00:19 | Níve
l=2 | ARP reply

--- Estatísticas ---
ARP       : 4
(.venv) root@AfonsoHenriques:/home/core/Desktop/RC# []
```

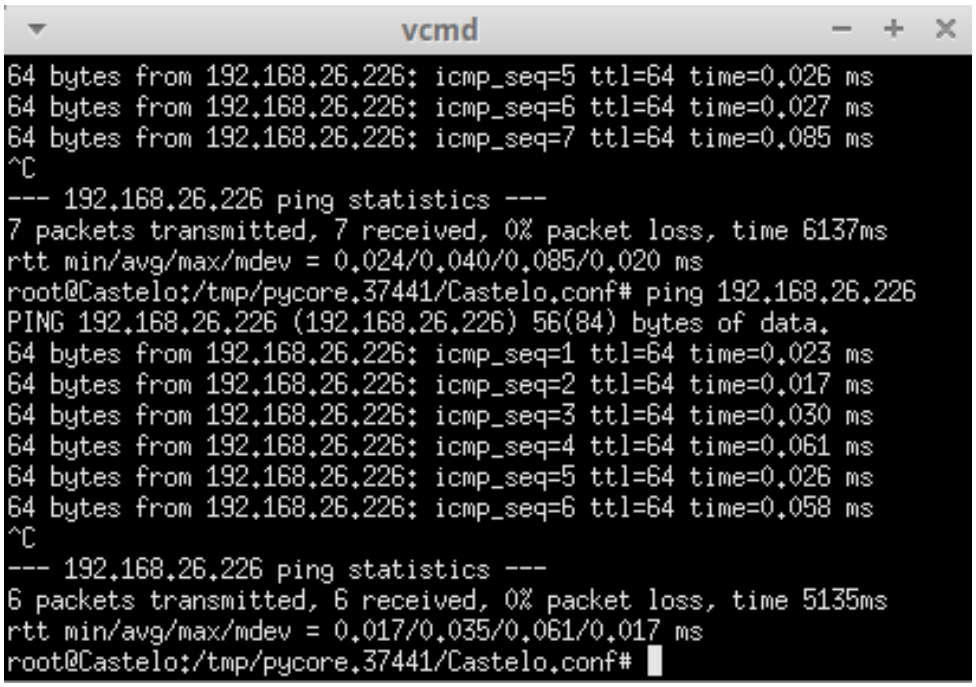
Figure 6: Captura de pacotes ARP no sniffer

Na Figura 6, o sniffer capturou mensagens *ARP request* seguidas das respetivas *ARP reply*. O pedido é enviado em broadcast, sendo recebido por todos os dispositivos da rede, enquanto a resposta é enviada diretamente ao emissor.

A sequência request/reply observada corresponde ao processo de resolução de endereços IP para MAC necessário antes da comunicação IP, demonstrando que o sniffer consegue identificar corretamente este tipo de interação.

## 3.5 Teste IPv4

Para observar tráfego ao nível da camada de rede, foi executado o comando *ping* entre dois nós, gerando pacotes encapsulados em IPv4.



```
vcmd
64 bytes from 192.168.26.226: icmp_seq=5 ttl=64 time=0.026 ms
64 bytes from 192.168.26.226: icmp_seq=6 ttl=64 time=0.027 ms
64 bytes from 192.168.26.226: icmp_seq=7 ttl=64 time=0.085 ms
^C
--- 192.168.26.226 ping statistics ---
7 packets transmitted, 7 received, 0% packet loss, time 6137ms
rtt min/avg/max/mdev = 0.024/0.040/0.085/0.020 ms
root@Castelo:/tmp/pycore.37441/Castelo.conf# ping 192.168.26.226
PING 192.168.26.226 (192.168.26.226) 56(84) bytes of data.
64 bytes from 192.168.26.226: icmp_seq=1 ttl=64 time=0.023 ms
64 bytes from 192.168.26.226: icmp_seq=2 ttl=64 time=0.017 ms
64 bytes from 192.168.26.226: icmp_seq=3 ttl=64 time=0.030 ms
64 bytes from 192.168.26.226: icmp_seq=4 ttl=64 time=0.061 ms
64 bytes from 192.168.26.226: icmp_seq=5 ttl=64 time=0.026 ms
64 bytes from 192.168.26.226: icmp_seq=6 ttl=64 time=0.058 ms
^C
--- 192.168.26.226 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5135ms
rtt min/avg/max/mdev = 0.017/0.035/0.061/0.017 ms
root@Castelo:/tmp/pycore.37441/Castelo.conf#
```

Figure 7: Execução do comando ping para gerar tráfego IPv4

Como se observa na Figura 7, a comunicação entre os nós foi bem-sucedida, sem perda de pacotes. Este tráfego é transportado sobre IPv4, sendo encapsulado com endereços IP de origem e destino.

```
(.venv) root@AfonsoHenriques:/home/core/Desktop/RC# sudo ./.venv/bin/python sniffer.py --iface eth0 --proto IPv4 --live --count 10
sudo: unable to resolve host AfonsoHenriques: Temporary failure in name resolution
WARNING: Could not retrieve the OS's nameserver !
TIME       SOURCE            DESTINATION       PROTO    INFO
--------------------------------------------------------------------------------
15:25:00   192.168.26.225    224.0.0.5         IPv4     capture(id=3) 192.168.26.225 -> 224.0.0.5 | Nível=3 | IPv4 packet
15:25:02   192.168.26.225    224.0.0.5         IPv4     capture(id=9) 192.168.26.225 -> 224.0.0.5 | Nível=3 | IPv4 packet

--- Estatísticas ---
IPv4      : 2
(.venv) root@AfonsoHenriques:/home/core/Desktop/RC#
```

Figure 8: Captura de pacotes IPv4 no sniffer

Na Figura 8, o sniffer apresenta apenas pacotes IPv4, uma vez que foi aplicado o filtro correspondente. É possível identificar os endereços IP de origem e destino, bem como o tipo de tráfego associado.

Destaca-se a presença de pacotes com destino multicast `224.0.0.5`, evidenciando que o sniffer consegue identificar diferentes tipos de comunicação ao nível da camada de rede.

A correspondência entre o tráfego gerado pelo *ping* e os pacotes capturados confirma que o filtro IPv4 funciona corretamente e que o sniffer interpreta adequadamente os campos relevantes desta camada.

## 3.6 Teste DHCP

```
<e.37441/AfonsoHenriques.conf# sudo dhclient  eth0
sudo: unable to resolve host AfonsoHenriques: Temporary failure in name resoluti
on
^C
```

Figure 9: Captura de mensagens DHCP Discover

Neste teste foi aplicado um filtro ao protocolo DHCP. Como se observa na Figura 9, foram capturadas mensagens *DHCP Discover* com origem `0.0.0.0` e destino `255.255.255.255`.

Este padrão é consistente com a fase *Discover* do protocolo DHCP, na qual o cliente ainda não possui endereço IP e envia um pedido em broadcast para localizar um servidor DHCP disponível.

```
(.venv) root@AfonsoHenriques:/home/core/Desktop/RC# sudo ./.venv/bin/python sniffer.py --iface eth0 --proto DHCP --live --count 10
sudo: unable to resolve host AfonsoHenriques: Temporary failure in name resolution
WARNING: Could not retrieve the OS's nameserver !
TIME       SOURCE            DESTINATION       PROTO    INFO
--------------------------------------------------------------------------------
14:59:14   0.0.0.0           255.255.255.255   DHCP     request(id=1) 0.0.0.0 -> 255.255.255.255 | Nível=4 ports=68->67 | DHCP Discover
14:59:17   0.0.0.0           255.255.255.255   DHCP     request(id=4) 0.0.0.0 -> 255.255.255.255 | Nível=4 ports=68->67 | DHCP Discover
14:59:20   0.0.0.0           255.255.255.255   DHCP     request(id=7) 0.0.0.0 -> 255.255.255.255 | Nível=4 ports=68->67 | DHCP Discover

--- Estatísticas ---
DHCP      : 3
```

Figure 10: Execução do comando dhclient

O tráfego observado foi gerado através do comando `dhclient`, que força a interface a solicitar um endereço IP.

## 3.7 Conclusão Parte A

Os testes realizados na Parte A demonstram que o sniffer funciona corretamente no ambiente CORE, sendo capaz de capturar pacotes, identificar protocolos e aplicar filtros de forma eficaz.

Em particular, foi possível observar a correspondência entre pedidos e respostas nos testes ICMP, o processo de resolução ARP e a fase inicial do protocolo DHCP através das mensagens *Discover*.

Estes resultados confirmam que a aplicação interpreta corretamente o comportamento dos protocolos analisados e cumpre os objetivos definidos para esta fase do trabalho.

# 4 Parte B

Nesta fase do trabalho, o sniffer foi executado numa interface de rede real, com o objetivo de validar a sua capacidade de capturar e analisar tráfego fora do ambiente controlado do CORE.

Foram realizados vários testes práticos que permitiram observar tráfego real, incluindo protocolos como ICMP, ARP, DHCP, DNS e HTTP/HTTPS, analisando os padrões de comunicação associados a cada um.

## 4.1 Teste ARP na interface real

```
| Resumo          |
| Interface       | wlp60s0
| Live            | sim
| Log             | -
| Formato         | json
| Proto           | ARP
| IP              | -
| MAC             | -
| BPF             | -
| HFilter         | -
| Count           | 0

Prima ENTER para voltar ao menu...

FILTROS DISPONÍVEIS

  1) Protocolo
  2) IP
  3) MAC
  4) BPF
  5) hfilter
  6) Limpar filtros
  7) Ver resumo
  8) Iniciar captura
  0) Voltar
Escolhe uma opção: 8

[Estado] Captura com filtros selecionada.
[Estado] Proto=ARP | IP=- | MAC=- | BPF=- | HFilter=-
TIME       SOURCE            DESTINATION        PROTO    INFO
--------------------------------------------------------------------------------
16:15:09   a4:6b:b6:3e:c7:5e  7c:39:85:62:05:43  ARP      request(id=86) a4:6b:b6:3e:c7:5e -> 7c:39:85:62:05:43 | Nível=2 | ARP request
16:15:09   7c:39:85:62:05:43  a4:6b:b6:3e:c7:5e  ARP      reply(id=92) ao request(id=86) 7c:39:85:62:05:43 -> a4:6b:b6:3e:c7:5e | Nível=2 | ARP reply
^C
--- Estatisticas ---
ARP       : 2
```

Figure 11: Captura de pacotes ARP na interface real

```
(base) prancha@Linux:~$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=19.5 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=117 time=18.8 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=117 time=18.3 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=117 time=19.2 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=117 time=18.9 ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=117 time=19.2 ms
64 bytes from 8.8.8.8: icmp_seq=7 ttl=117 time=20.1 ms
64 bytes from 8.8.8.8: icmp_seq=8 ttl=117 time=19.3 ms
64 bytes from 8.8.8.8: icmp_seq=9 ttl=117 time=19.6 ms
64 bytes from 8.8.8.8: icmp_seq=10 ttl=117 time=20.0 ms
64 bytes from 8.8.8.8: icmp_seq=11 ttl=117 time=19.4 ms
64 bytes from 8.8.8.8: icmp_seq=12 ttl=117 time=18.8 ms
64 bytes from 8.8.8.8: icmp_seq=13 ttl=117 time=18.9 ms
64 bytes from 8.8.8.8: icmp_seq=14 ttl=117 time=18.8 ms
64 bytes from 8.8.8.8: icmp_seq=15 ttl=117 time=19.0 ms
64 bytes from 8.8.8.8: icmp_seq=16 ttl=117 time=18.4 ms
^C
--- 8.8.8.8 ping statistics ---
16 packets transmitted, 16 received, 0% packet loss, time 15022ms
rtt min/avg/max/mdev = 18.327/19.139/20.131/0.500 ms
```

Figure 12: Execução do comando ping para 8.8.8.8

Como se observa na Figura 4.1, foram capturadas mensagens *ARP request* seguidas das respetivas *ARP reply*. O pedido é enviado em broadcast, sendo recebido por todos os dispositivos da rede local, enquanto a resposta é enviada diretamente ao emissor.

A sequência request/reply observada corresponde ao processo de resolução de endereços IP para MAC, demonstrando que o sniffer consegue identificar corretamente este tipo de interação em ambiente real.

## 4.2 Filtragem de tráfego ICMP

```
| Resumo      |
| Interface   | wlp60s0
| Live        | sim
| Log         | -
| Formato     | json
| Proto       | ICMP
| IP          | -
| MAC         | -
| BPF         | -
| HFilter     | -
| Count       | 0

Prima ENTER para voltar ao menu...

FILTROS DISPONÍVEIS

 1) Protocolo
 2) IP
 3) MAC
 4) BPF
 5) hfilter
 ──────────────────
 6) Limpar filtros
 7) Ver resumo
 ──────────────────
 8) Iniciar captura
 0) Voltar
Escolhe uma opção: 8

[Estado] Captura com filtros selecionada.
[Estado] Proto=ICMP | IP=- | MAC=- | BPF=- | HFilter=-
TIME       SOURCE          DESTINATION       PROTO    INFO
--------------------------------------------------------------------------------
16:17:19   192.168.1.159   8.8.8.8           ICMP     request(id=1) 192.168.1.159 -> 8.8.8.8 | Nível=4 | ICMP echo request
16:17:19   8.8.8.8         192.168.1.159     ICMP     reply(id=2) ao request(id=1) 8.8.8.8 -> 192.168.1.159 | Nível=4 | ICMP echo reply
16:17:20   192.168.1.159   8.8.8.8           ICMP     request(id=11) 192.168.1.159 -> 8.8.8.8 | Nível=4 | ICMP echo request
16:17:20   8.8.8.8         192.168.1.159     ICMP     reply(id=12) ao request(id=11) 8.8.8.8 -> 192.168.1.159 | Nível=4 | ICMP echo reply
16:17:21   192.168.1.159   8.8.8.8           ICMP     request(id=42) 192.168.1.159 -> 8.8.8.8 | Nível=4 | ICMP echo request
16:17:21   8.8.8.8         192.168.1.159     ICMP     reply(id=43) ao request(id=42) 8.8.8.8 -> 192.168.1.159 | Nível=4 | ICMP echo reply
16:17:22   192.168.1.159   8.8.8.8           ICMP     request(id=58) 192.168.1.159 -> 8.8.8.8 | Nível=4 | ICMP echo request
16:17:22   8.8.8.8         192.168.1.159     ICMP     reply(id=60) ao request(id=58) 8.8.8.8 -> 192.168.1.159 | Nível=4 | ICMP echo reply
16:17:23   192.168.1.159   8.8.8.8           ICMP     request(id=74) 192.168.1.159 -> 8.8.8.8 | Nível=4 | ICMP echo request
16:17:23   8.8.8.8         192.168.1.159     ICMP     reply(id=75) ao request(id=74) 8.8.8.8 -> 192.168.1.159 | Nível=4 | ICMP echo reply
16:17:24   192.168.1.159   8.8.8.8           ICMP     request(id=81) 192.168.1.159 -> 8.8.8.8 | Nível=4 | ICMP echo request
16:17:24   8.8.8.8         192.168.1.159     ICMP     reply(id=85) ao request(id=81) 8.8.8.8 -> 192.168.1.159 | Nível=4 | ICMP echo reply
^C
--- Estatísticas ---
ICMP      : 12
```

Figure 13: Captura de tráfego ICMP com filtro aplicado

```
(base) prancha@Linux:~$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=19.4 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=117 time=19.1 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=117 time=18.6 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=117 time=19.1 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=117 time=18.8 ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=117 time=20.1 ms
64 bytes from 8.8.8.8: icmp_seq=7 ttl=117 time=20.1 ms
64 bytes from 8.8.8.8: icmp_seq=8 ttl=117 time=19.0 ms
^C
--- 8.8.8.8 ping statistics ---
8 packets transmitted, 8 received, 0% packet loss, time 7010ms
rtt min/avg/max/mdev = 18.606/19.266/20.143/0.529 ms
```

Figure 14: Execução do comando ping para 8.8.8.8

Neste teste foi executado o comando `ping 8.8.8.8` para gerar tráfego ICMP. Como se observa na Figura 14, não houve perda de pacotes e os valores de RTT situam-se entre aproximadamente 18 ms e 21 ms, refletindo uma comunicação através da Internet.

Na Figura 13, o sniffer capturou mensagens *ICMP echo request* enviadas pelo host local e as respetivas *echo reply*. Observa-se que cada pedido gera uma resposta correspondente, sendo possível identificar pares request/reply através do identificador interno do sniffer.

A correspondência direta entre estes pacotes confirma o comportamento observado na execução do *ping* e demonstra que o sniffer consegue reconstruir a sequência de comunicação entre os dois nós.

## 4.3 Teste IPv4 na interface real

```
| Resumo          |
| Interface       | wlp60s0
| Live            | sim
| Log             | -
| Formato         | json
| Proto           | IPv4
| IP              | -
| MAC             | -
| BPF             | -
| HFilter         | -
| Count           | 0

Prima ENTER para voltar ao menu...

FILTROS DISPONÍVEIS

 1) Protocolo
 2) IP
 3) MAC
 4) BPF
 5) hfilter
 6) Count
 ──────────────────────
 7) Limpar filtros
 8) Ver resumo
 ──────────────────────
 9) Iniciar captura
 0) Voltar
Escolhe uma opção: 9

[Estado] Captura com filtros selecionada.
[Estado] Proto=IPv4 | IP=- | MAC=- | BPF=- | HFilter=-
TIME        SOURCE            DESTINATION       PROTO    INFO
--------------------------------------------------------------------------------
17:07:53   192.168.1.159      8.8.8.8           IPv4     request(id=4) 192.168.1.159 -> 8.8.8.8 | Nível=4 | ICMP echo request
17:07:53   8.8.8.8            192.168.1.159     IPv4     reply(id=5) ao request(id=4) 8.8.8.8 -> 192.168.1.159 | Nível=4 | ICMP echo reply
17:07:54   192.168.1.159      8.8.8.8           IPv4     request(id=7) 192.168.1.159 -> 8.8.8.8 | Nível=4 | ICMP echo request
17:07:54   8.8.8.8            192.168.1.159     IPv4     reply(id=8) ao request(id=7) 8.8.8.8 -> 192.168.1.159 | Nível=4 | ICMP echo reply
17:07:55   192.168.1.159      8.8.8.8           IPv4     request(id=9) 192.168.1.159 -> 8.8.8.8 | Nível=4 | ICMP echo request
17:07:55   8.8.8.8            192.168.1.159     IPv4     reply(id=10) ao request(id=9) 8.8.8.8 -> 192.168.1.159 | Nível=4 | ICMP echo reply
17:07:56   192.168.1.159      8.8.8.8           IPv4     request(id=11) 192.168.1.159 -> 8.8.8.8 | Nível=4 | ICMP echo request
17:07:56   8.8.8.8            192.168.1.159     IPv4     reply(id=12) ao request(id=11) 8.8.8.8 -> 192.168.1.159 | Nível=4 | ICMP echo reply
17:07:57   192.168.1.159      8.8.8.8           IPv4     request(id=13) 192.168.1.159 -> 8.8.8.8 | Nível=4 | ICMP echo request
17:07:57   8.8.8.8            192.168.1.159     IPv4     reply(id=14) ao request(id=13) 8.8.8.8 -> 192.168.1.159 | Nível=4 | ICMP echo reply
17:07:57   192.168.1.159      140.82.113.21     IPv4     capture(id=16) 192.168.1.159 -> 140.82.113.21 | Nível=4 ports=43538->443 | HTTP request 43538 -> 4
43
17:07:57   140.82.113.21      192.168.1.159     IPv4     capture(id=17) 140.82.113.21 -> 192.168.1.159 | Nível=4 ports=443->43538 | HTTP response 443 -> 43
538
17:07:58   192.168.1.159      8.8.8.8           IPv4     request(id=18) 192.168.1.159 -> 8.8.8.8 | Nível=4 | ICMP echo request
17:07:58   8.8.8.8            192.168.1.159     IPv4     reply(id=19) ao request(id=18) 8.8.8.8 -> 192.168.1.159 | Nível=4 | ICMP echo reply
17:07:59   192.168.1.159      8.8.8.8           IPv4     request(id=24) 192.168.1.159 -> 8.8.8.8 | Nível=4 | ICMP echo request
17:07:59   8.8.8.8            192.168.1.159     IPv4     reply(id=25) ao request(id=24) 8.8.8.8 -> 192.168.1.159 | Nível=4 | ICMP echo reply
17:08:00   192.168.1.159      8.8.8.8           IPv4     request(id=40) 192.168.1.159 -> 8.8.8.8 | Nível=4 | ICMP echo request
17:08:00   8.8.8.8            192.168.1.159     IPv4     reply(id=41) ao request(id=40) 8.8.8.8 -> 192.168.1.159 | Nível=4 | ICMP echo reply
```

Figure 15: Captura de mensagens IPv4 com filtro aplicado

```
(base) prancha@Linux:~$ ping -c 20 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=19.7 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=117 time=20.4 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=117 time=21.4 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=117 time=20.9 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=117 time=19.2 ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=117 time=19.2 ms
64 bytes from 8.8.8.8: icmp_seq=7 ttl=117 time=21.2 ms
64 bytes from 8.8.8.8: icmp_seq=8 ttl=117 time=18.9 ms
64 bytes from 8.8.8.8: icmp_seq=9 ttl=117 time=19.2 ms
64 bytes from 8.8.8.8: icmp_seq=10 ttl=117 time=19.3 ms
64 bytes from 8.8.8.8: icmp_seq=11 ttl=117 time=19.4 ms
64 bytes from 8.8.8.8: icmp_seq=12 ttl=117 time=19.6 ms
64 bytes from 8.8.8.8: icmp_seq=13 ttl=117 time=19.2 ms
64 bytes from 8.8.8.8: icmp_seq=14 ttl=117 time=18.0 ms
64 bytes from 8.8.8.8: icmp_seq=15 ttl=117 time=20.2 ms
^C
--- 8.8.8.8 ping statistics ---
15 packets transmitted, 15 received, 0% packet loss, time 14023ms
rtt min/avg/max/mdev = 17.989/19.720/21.386/0.899 ms
```

Figure 16: Execução do comando ping para 8.8.8.8

Para validar a captura de tráfego ao nível da camada de rede numa interface real, foi aplicado o filtro IPv4 ao sniffer e executado o comando `ping 8.8.8.8`, conforme ilustrado na Figura 16.

Como se observa na Figura 15, com o filtro IPv4 ativo, o sniffer apresenta exclusivamente pacotes ao nível da camada de rede. É possível identificar os endereços IP de origem e destino em cada pacote, bem como os diferentes tipos de tráfego encapsulados, nomeadamente mensagens ICMP geradas pelo *ping* e tráfego HTTP associado a comunicações com servidores externos.

Ao contrário do ambiente CORE, numa interface real o tráfego IPv4 é significativamente mais heterogéneo, surgindo simultaneamente pacotes de diferentes aplicações e serviços em execução no sistema. Esta diversidade confirma a capacidade do sniffer para capturar e classificar tráfego ao nível da camada de rede em condições reais de utilização.

## 4.4 Teste DHCP na interface real

```
| Resumo |
| Interface | wlp60s0 |
| Live      | sim     |
| Log       | -       |
| Formato   | json    |
| Proto     | DHCP    |
| IP        | -       |
| MAC       | -       |
| BPF       | -       |
| HFilter   | -       |
| Count     | 0       |

Prima ENTER para voltar ao menu...

FILTROS DISPONÍVEIS

 1) Protocolo
 2) IP
 3) MAC
 4) BPF
 5) hfilter
 6) Count
 7) Limpar filtros
 8) Ver resumo
 9) Iniciar captura
 0) Voltar
Escolhe uma opção: 9

[Estado] Captura com filtros selecionada.
[Estado] Proto=DHCP | IP=- | MAC=- | BPF=- | HFilter=-
TIME      SOURCE          DESTINATION     PROTO   INFO
--------------------------------------------------------------------------------
11:55:35  0.0.0.0         255.255.255.255 DHCP    request(id=180) 0.0.0.0 -> 255.255.255.255 | Nível=4 ports=68->67 | DHCP Discover
11:55:36  1.1.1.10        172.26.231.45   DHCP    reply(id=181) ao request(id=180) 1.1.1.10 -> 172.26.231.45 | Nível=4 ports=67->68 | DHCP Offer
11:55:36  0.0.0.0         255.255.255.255 DHCP    request(id=183) 0.0.0.0 -> 255.255.255.255 | Nível=4 ports=68->67 | DHCP Request
11:55:36  1.1.1.10        172.26.231.45   DHCP    reply(id=184) ao request(id=183) 1.1.1.10 -> 172.26.231.45 | Nível=4 ports=67->68 | DHCP Ack
^C
--- Estatísticas ---
DHCP      : 4
```

Figure 17: Captura de mensagens DHCP

Neste teste foi utilizado o comando `dhclient` para gerar tráfego DHCP numa interface real.

Como se observa na Figura 17, é possível identificar a sequência de mensagens *Discover*, *Offer*, *Request* e *Acknowledge*. Inicialmente, o cliente envia um pedido em broadcast (`0.0.0.0` → `255.255.255.255`), sendo posteriormente respondido pelo servidor com uma proposta de configuração.

De seguida, o cliente envia um *Request* para aceitar a proposta, ao qual o servidor responde com um *Acknowledge*. Esta sequência confirma que o sniffer consegue identificar corretamente as diferentes fases do processo DHCP e interpretar a interação completa entre cliente e servidor.

## 4.5 Captura de tráfego sem filtros

```
[Estado] Captura sem filtros selecionada.
TIME       SOURCE              DESTINATION         PROTO   INFO
--------------------------------------------------------------------------------
11:59:54   172.26.22.44        162.247.243.29      HTTP    capture(id=1) 172.26.22.44 -> 162.247.243.29 | Nível=4 ports=42390->443 | HTTP request 42390 -> 443
11:59:54   162.247.243.29      172.26.22.44        HTTP    capture(id=2) 162.247.243.29 -> 172.26.22.44 | Nível=4 ports=443->42390 | HTTP response 443 -> 42390
12:00:02   3.125.250.70        172.26.22.44        HTTP    capture(id=3) 3.125.250.70 -> 172.26.22.44 | Nível=4 ports=443->49058 | HTTP response 443 -> 49058
12:00:02   172.26.22.44        3.125.250.70        HTTP    capture(id=4) 172.26.22.44 -> 3.125.250.70 | Nível=4 ports=49058->443 | HTTP request 49058 -> 443
12:00:02   172.26.22.44        3.125.250.70        HTTP    capture(id=5) 172.26.22.44 -> 3.125.250.70 | Nível=4 ports=49058->443 | HTTP request 49058 -> 443
12:00:02   3.125.250.70        172.26.22.44        HTTP    capture(id=6) 3.125.250.70 -> 172.26.22.44 | Nível=4 ports=443->49058 | HTTP response 443 -> 49058
12:00:03   172.26.22.44        162.247.243.29      HTTP    capture(id=7) 172.26.22.44 -> 162.247.243.29 | Nível=4 ports=42390->443 | HTTP request 42390 -> 443
12:00:03   162.247.243.29      172.26.22.44        HTTP    capture(id=8) 162.247.243.29 -> 172.26.22.44 | Nível=4 ports=443->42390 | HTTP response 443 -> 42390
12:00:03   162.247.243.29      172.26.22.44        HTTP    capture(id=9) 162.247.243.29 -> 172.26.22.44 | Nível=4 ports=443->42390 | HTTP response 443 -> 42390
12:00:03   172.26.22.44        162.247.243.29      HTTP    capture(id=10) 172.26.22.44 -> 162.247.243.29 | Nível=4 ports=42390->443 | HTTP request 42390 -> 443
12:00:03   162.247.243.29      172.26.22.44        HTTP    capture(id=11) 162.247.243.29 -> 172.26.22.44 | Nível=4 ports=443->42390 | HTTP response 443 -> 42390
12:00:03   172.26.22.44        162.247.243.29      HTTP    capture(id=12) 172.26.22.44 -> 162.247.243.29 | Nível=4 ports=42390->443 | HTTP request 42390 -> 443
12:00:03   162.247.243.29      172.26.22.44        HTTP    capture(id=13) 162.247.243.29 -> 172.26.22.44 | Nível=4 ports=443->42390 | HTTP response 443 -> 42390
12:00:03   172.26.22.44        162.247.243.29      HTTP    capture(id=14) 172.26.22.44 -> 162.247.243.29 | Nível=4 ports=42390->443 | HTTP request 42390 -> 443
12:00:03   172.26.22.44        162.247.243.29      HTTP    capture(id=15) 172.26.22.44 -> 162.247.243.29 | Nível=4 ports=42390->443 | HTTP request 42390 -> 443
12:00:03   162.247.243.29      172.26.22.44        HTTP    capture(id=16) 162.247.243.29 -> 172.26.22.44 | Nível=4 ports=443->42390 | HTTP response 443 -> 42390
12:00:03   162.247.243.29      172.26.22.44        HTTP    capture(id=17) 162.247.243.29 -> 172.26.22.44 | Nível=4 ports=443->42390 | HTTP response 443 -> 42390
12:00:03   172.26.22.44        162.247.241.14      HTTP    capture(id=18) 172.26.22.44 -> 162.247.241.14 | Nível=4 ports=37430->443 | HTTP request 37430 -> 443
12:00:03   162.247.241.14      172.26.22.44        HTTP    capture(id=19) 162.247.241.14 -> 172.26.22.44 | Nível=4 ports=443->37430 | HTTP response 443 -> 37430
12:00:03   162.247.241.14      172.26.22.44        HTTP    capture(id=20) 162.247.241.14 -> 172.26.22.44 | Nível=4 ports=443->37430 | HTTP response 443 -> 37430
12:00:03   172.26.22.44        162.247.241.14      HTTP    capture(id=21) 172.26.22.44 -> 162.247.241.14 | Nível=4 ports=37430->443 | HTTP request 37430 -> 443
12:00:13   172.26.22.44        162.247.241.14      HTTP    capture(id=22) 172.26.22.44 -> 162.247.241.14 | Nível=4 ports=37430->443 | HTTP request 37430 -> 443
12:00:14   162.247.241.14      172.26.22.44        HTTP    capture(id=23) 162.247.241.14 -> 172.26.22.44 | Nível=4 ports=443->37430 | HTTP response 443 -> 37430
12:00:14   fe80::27c4:b0f1:2806:a4ab ff02::fb          DNS     request(id=24) fe80::27c4:b0f1:2806:a4ab -> ff02::fb | Nível=4 ports=5353->5353 | DNS query _ftp._tcp.local
12:00:14   172.26.22.44        224.0.0.251         DNS     request(id=25) 172.26.22.44 -> 224.0.0.251 | Nível=4 ports=5353->5353 | DNS query _ftp._tcp.local
12:00:17   172.26.22.44        193.137.16.75       DNS     request(id=26) 172.26.22.44 -> 193.137.16.75 | Nível=4 ports=39199->53 | DNS query mail.google.com
12:00:17   172.26.22.44        193.137.16.75       DNS     request(id=27) 172.26.22.44 -> 193.137.16.75 | Nível=4 ports=55813->53 | DNS query mail.google.com
12:00:17   193.137.16.75       172.26.22.44        DNS     reply(id=28) ao request(id=27) 193.137.16.75 -> 172.26.22.44 | Nível=4 ports=53->55813 | DNS response mail.google.com
12:00:17   193.137.16.75       172.26.22.44        DNS     reply(id=29) ao request(id=26) 193.137.16.75 -> 172.26.22.44 | Nível=4 ports=53->39199 | DNS response mail.google.com
12:00:17   172.26.22.44        142.251.142.197     HTTP    capture(id=30) 172.26.22.44 -> 142.251.142.197 | Nível=4 ports=52714->443 | HTTP request 52714 -> 443
12:00:17   142.251.142.197     172.26.22.44        HTTP    capture(id=31) 142.251.142.197 -> 172.26.22.44 | Nível=4 ports=443->52714 | HTTP response 443 -> 52714
12:00:17   172.26.22.44        142.251.142.197     HTTP    capture(id=32) 172.26.22.44 -> 142.251.142.197 | Nível=4 ports=52714->443 | HTTP request 52714 -> 443
12:00:17   172.26.22.44        142.251.142.197     HTTP    capture(id=33) 172.26.22.44 -> 142.251.142.197 | Nível=4 ports=52714->443 | HTTP request 52714 -> 443
12:00:17   172.26.22.44        142.251.142.197     HTTP    capture(id=34) 172.26.22.44 -> 142.251.142.197 | Nível=4 ports=52722->443 | HTTP request 52722 -> 443
12:00:17   142.251.142.197     172.26.22.44        HTTP    capture(id=35) 142.251.142.197 -> 172.26.22.44 | Nível=4 ports=443->52714 | HTTP response 443 -> 52714
12:00:17   142.251.142.197     172.26.22.44        HTTP    capture(id=36) 142.251.142.197 -> 172.26.22.44 | Nível=4 ports=443->52714 | HTTP response 443 -> 52714
12:00:17   172.26.22.44        142.251.142.197     HTTP    capture(id=37) 172.26.22.44 -> 142.251.142.197 | Nível=4 ports=52714->443 | HTTP request 52714 -> 443
12:00:17   142.251.142.197     172.26.22.44        HTTP    capture(id=38) 142.251.142.197 -> 172.26.22.44 | Nível=4 ports=443->52714 | HTTP response 443 -> 52714
12:00:17   172.26.22.44        142.251.142.197     HTTP    capture(id=39) 172.26.22.44 -> 142.251.142.197 | Nível=4 ports=52714->443 | HTTP request 52714 -> 443
12:00:17   142.251.142.197     172.26.22.44        HTTP    capture(id=40) 142.251.142.197 -> 172.26.22.44 | Nível=4 ports=443->52714 | HTTP response 443 -> 52714
12:00:17   172.26.22.44        142.251.142.197     HTTP    capture(id=41) 172.26.22.44 -> 142.251.142.197 | Nível=4 ports=52714->443 | HTTP request 52714 -> 443
12:00:17   142.251.142.197     172.26.22.44        HTTP    capture(id=42) 142.251.142.197 -> 172.26.22.44 | Nível=4 ports=443->52714 | HTTP response 443 -> 52714
12:00:17   172.26.22.44        142.251.142.197     HTTP    capture(id=43) 172.26.22.44 -> 142.251.142.197 | Nível=4 ports=52714->443 | HTTP request 52714 -> 443
12:00:17   172.26.22.44        142.251.142.197     HTTP    capture(id=44) 172.26.22.44 -> 142.251.142.197 | Nível=4 ports=52714->443 | HTTP request 52714 -> 443
12:00:17   172.26.22.44        142.251.142.197     HTTP    capture(id=45) 172.26.22.44 -> 142.251.142.197 | Nível=4 ports=52714->443 | HTTP request 52714 -> 443
12:00:17   142.251.142.197     172.26.22.44        HTTP    capture(id=46) 142.251.142.197 -> 172.26.22.44 | Nível=4 ports=443->52722 | HTTP response 443 -> 52722
12:00:17   172.26.22.44        142.251.142.197     HTTP    capture(id=47) 172.26.22.44 -> 142.251.142.197 | Nível=4 ports=52722->443 | HTTP request 52722 -> 443
12:00:17   142.251.142.197     172.26.22.44        HTTP    capture(id=48) 142.251.142.197 -> 172.26.22.44 | Nível=4 ports=443->52714 | HTTP response 443 -> 52714
12:00:17   172.26.22.44        142.251.142.197     HTTP    capture(id=49) 172.26.22.44 -> 142.251.142.197 | Nível=4 ports=52714->443 | HTTP request 52714 -> 443
12:00:17   142.251.142.197     172.26.22.44        HTTP    capture(id=50) 142.251.142.197 -> 172.26.22.44 | Nível=4 ports=443->52714 | HTTP response 443 -> 52714

--- Estatísticas ---
HTTP      : 44
DNS       : 6
```

Figure 18: Captura de tráfego sem filtros na interface real

Neste teste o sniffer foi executado sem qualquer filtro ativo, com o objetivo de observar o tráfego real gerado durante a utilização normal do sistema. Como se verifica na Figura 18, são capturadas múltiplas comunicações associadas a tráfego web na porta 443, correspondente a HTTPS, bem como pedidos e respostas DNS.

Relativamente ao tráfego DNS, observa-se que cada *query* é seguida pela respetiva resposta, sendo possível identificar a correspondência entre ambos através dos identificadores de transação apresentados pelo sniffer. Esta correlação automática confirma o correto funcionamento do mecanismo de associação request/reply implementado para o protocolo DNS.

No caso do tráfego HTTPS, verificam-se múltiplas comunicações entre o cliente e diferentes servidores externos. Uma vez que o conteúdo aplicacional é transportado sobre TLS, o sniffer não tem acesso aos dados cifrados, identificando apenas metadados como endereços IP, portas e direção dos fluxos. Este comportamento é esperado e representa uma limitação intrínseca de qualquer analisador passivo de tráfego perante comunicações cifradas.

A captura demonstra ainda a capacidade do sniffer para identificar e distinguir múltiplos protocolos em simultâneo, classificando corretamente o tráfego heterogéneo presente numa interface real durante utilização normal.

## 4.6 Geração de gráficos

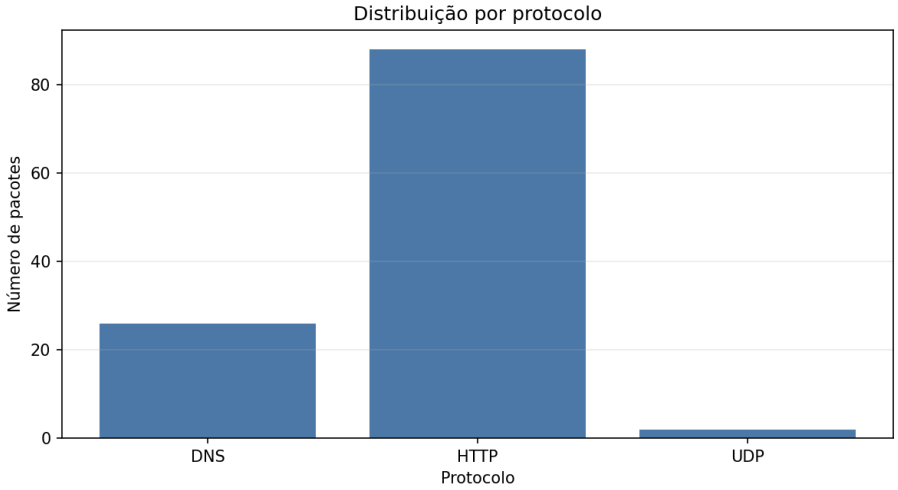



Figure 19: Gráfico de distribuição de pacotes por protocolo

A Figura 19 apresenta a distribuição dos pacotes capturados por protocolo. Observa-se que o tráfego HTTP/HTTPS é dominante nesta captura, seguido por DNS, enquanto UDP surge em menor quantidade. Este resultado é consistente com uma captura realizada numa interface real durante utilização normal da rede, onde aplicações web geram múltiplas comunicações com servidores externos.

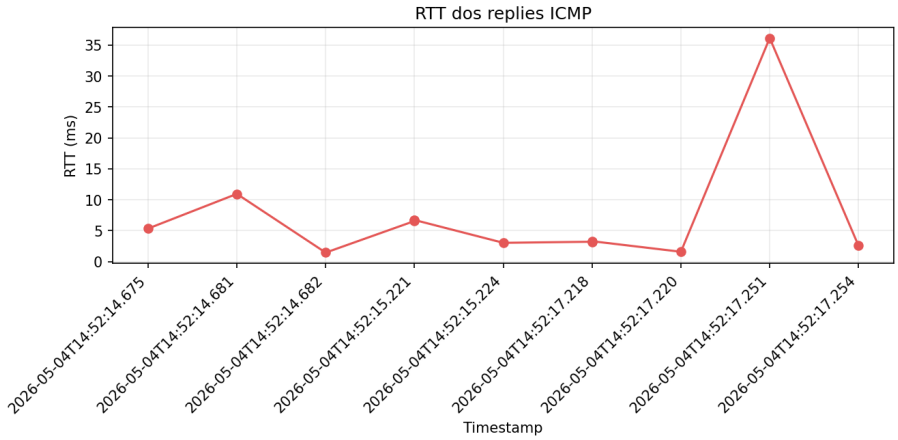



Figure 20: Gráfico de RTT dos *replies* ICMP ao longo do tempo

A Figura 20 mostra a evolução do RTT dos *replies* ICMP ao longo do tempo. Os valores mantêm-se maioritariamente baixos, com uma variação pontual mais elevada, o que permite observar oscilações de latência durante a comunicação.

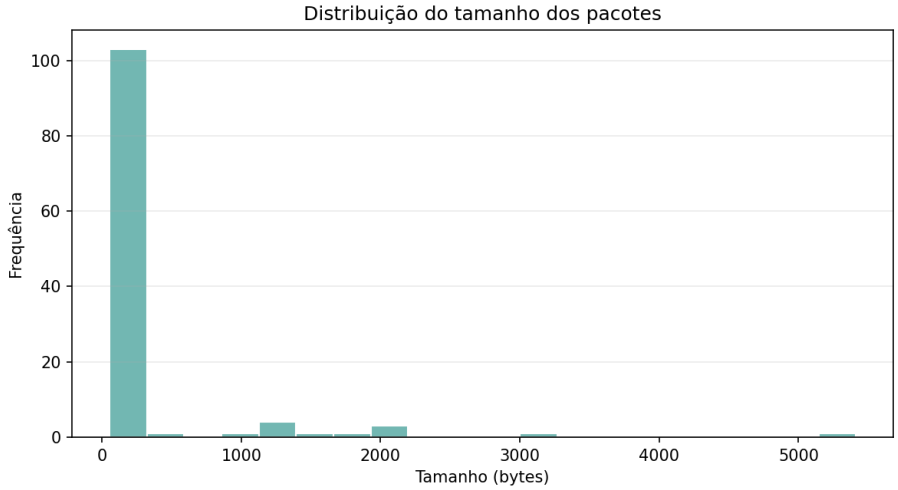



Figure 21: Distribuição do tamanho dos pacotes capturados

A Figura 21 apresenta a distribuição do tamanho dos pacotes capturados. Verifica-se que a maioria dos pacotes possui tamanho reduzido, enquanto alguns pacotes de maior dimensão surgem com menor frequência, refletindo a diferença entre pacotes de controlo e pacotes associados a transferência de dados.

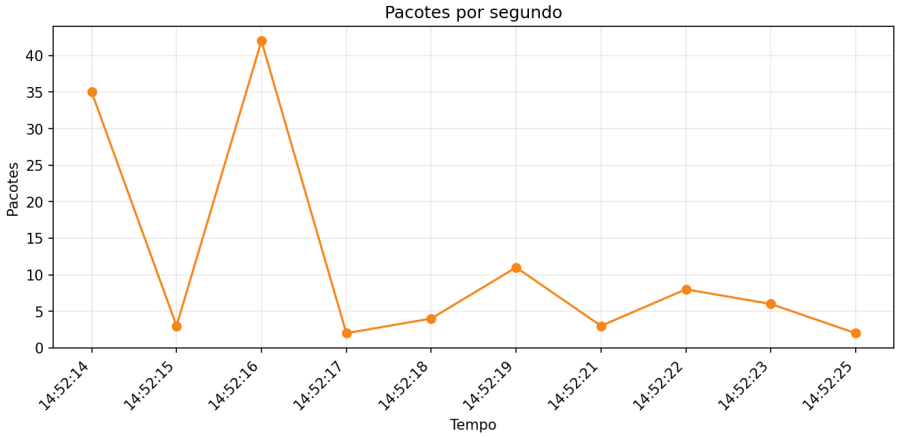



Figure 22: Número de pacotes capturados por segundo

Por fim, a Figura 22 representa o número de pacotes capturados por segundo. A variação observada indica que o tráfego não é constante ao longo da captura, existindo momentos com maior atividade, associados a rajadas de comunicação geradas pelas aplicações em execução.

## 4.7 Conclusão Parte B

Os testes realizados na interface real demonstram que o sniffer é capaz de capturar e analisar tráfego real, identificando corretamente múltiplos protocolos e os respetivos padrões de interação.

Foi possível observar sequências completas de comunicação, como pares request/reply em ICMP e DNS e o processo de atribuição dinâmica no DHCP, confirmando a capacidade do sniffer para interpretar o comportamento dos protocolos em condições reais de rede.

# 5 Conclusão

O desenvolvimento do packet sniffer permitiu implementar e validar um sistema de captura e análise de tráfego de rede capaz de identificar múltiplos protocolos, aplicar filtros e apresentar informação relevante sobre cada pacote capturado.

Na Parte A, em ambiente controlado no CORE, foi possível validar o funcionamento do sniffer de forma isolada, observando comportamentos específicos de protocolos como ICMP, ARP, IPv4 e DHCP. Já na Parte B, a execução numa interface real permitiu analisar tráfego mais diversificado, incluindo DNS e tráfego web, confirmando a capacidade da aplicação para operar em cenários reais.

A comparação entre os dois ambientes demonstra que o sniffer consegue capturar, classificar e interpretar pacotes em contextos distintos. Como limitação principal, destaca-se o facto de a visibilidade estar dependente da interface onde a aplicação é executada e de tráfego cifrado, como HTTPS, não permitir a análise do conteúdo aplicacional. Ainda assim, os resultados obtidos confirmam o cumprimento dos objetivos do trabalho e demonstram a correta implementação dos mecanismos de captura, parsing, filtragem, logging e análise de pacotes.